

Why two threads don't halve the commutator kernel – measured, not theorized

Status: historical investigation (2026-08-31), retained for its method. The measured lesson – small-call parallelism is decided by the serial prologue and sweep efficiency, not dependencies – still holds and is generalized in `parallelism-in-the-fold.typ` (work-adaptive scheduling). The specific mechanisms, baselines (`scratch`), and the rejected fix predate the class-contiguous layout, write-access classes, the single-phase per-unit sweep, and the caching layers; do not use its numbers to describe current code.

lie-rs – investigation report – 2026-08-31. Every number below is a fresh measurement of the current code (criterion baseline `scratch`); a proposed fix was implemented, measured, and rejected on this evidence.

Conclusion: the sweep's per-entry cost is **uniform** (≈ 7.5 ns/entry, independent of a pair's degree) – the existing entry-count bundling already balances the sweep in the metric that matters. The two-thread deficits are measured coordination costs: a per-fork round-trip ($\approx 0.9\text{--}1.4$ μ s) that exceeds the **entire** sweep of the smallest shapes, and the parallel path's extra serial work (flatten + bundle + bridge: $2.4\times$ the CPU cycles of the serial path). Work-balanced bundling was implemented, measured 5–8% **slower**, and reverted.

What the fresh measurements say (all seven shapes, 1/2/4/8 threads)

shape	1t	2t	4t	8t	$\times 2t$
3 $\times$ 8	21.5	16.4	16.1	18.5	1.31 $\times$
2 $\times$ 12	13.6	11.7	11.7	12.7	1.16 $\times$
4 $\times$ 6	13.8	13.1	12.7	14.3	1.05 $\times$
5 $\times$ 5	11.2	11.1	11.6	14.8	1.01 $\times$
6 $\times$ 4	5.2	7.8	8.8	10.5	0.66 $\times$
8 $\times$ 3	2.4	6.2	7.1	9.0	0.39 $\times$
12 $\times$ 2	0.87	3.5	4.6	5.6	0.25 $\times$

μ s per kernel call, criterion medians. Two regimes: shapes with sweeps ≥ 10 μ s gain partially and then flatline; shapes with sweeps under 2 μ s get **monotonically worse as threads are added** – 12 $\times$ 2 loses $4\times$ at two threads and keeps losing.

The small-shape regime: dispatch costs that exist only in the parallel regime

Two threads taking $4\times$ the time of one cannot be explained by work imbalance (that is bounded by the serial time) – the extra time must be coordination that only exists when a second worker exists. Each candidate was tested directly on 12 $\times$ 2:

- **Result-vector false sharing** – ruled out: the batch API gives every job its own result buffer; 64 disjoint buffers at 2 threads are still $1.66\times$ slower than 1 (84 vs 51 μ s per dispatch). Sharing the output is not the cause.
- **SMT co-location** – ruled out: pinning the workers to distinct physical cores reproduces free scheduling exactly (151.7 vs 151.6 μ s); forced SMT siblings are worse (192.7 μ s) – placement matters, but is not the cause.
- **Frequency** (MPERF/APERF: ≈ 3.4 GHz everywhere), **page faults**, **steal/wake latency** (park/steal symbols 2–3% of samples) – all minor.

What remains, measured. (1) A fixed per-fork coordination cost: forking and joining a 4-bundle `par_iter` with a spinning second worker costs $\approx 0.9\text{--}1.4$ μ s (empty-bundle micro-benchmark, net of the install handshake; it grows with worker count, which is why the degradation is monotone in threads). On 12 $\times$ 2 the **entire kernel** is 0.87 μ s and the sweep inside it ≈ 0.4 μ s – less than one fork. (2) The parallel path does materially more work than the serial one: the 1-thread path sweeps units directly and never enters rayon, while the multi-thread path flattens every (job, unit) pair into a task vector, copies it into heap-allocated bundles, and dispatches through rayon's bridge. Profiling 12 $\times$ 2 dispatches: 2 threads burn $2.4\times$ the CPU cycles for identical output (693 M vs 290 M), execute $1.8\times$ the instructions, and 45% of 2-thread samples land in bridge code the serial path never runs. This overhead scales with **jobs $\times$ units**, not with sweep time.

The large-sweep regime: the imbalance hypothesis, implemented and falsified

For 3 $\times$ 8 the earlier hardware-counter data (AMD IBS) showed the two workers splitting the sweep 52 : 48 in loads but 65 : 35 in **stores**, with store volume growing with a pair's degree – suggesting entry-count bundling under-balances the expensive high-degree pairs. That hypothesis was implemented: per-segment scatter work Ω (the telescoped decomposition-word count, `decomp_start[span_end-1] - decomp_start[span_start]`), bundles cut at equal Ω . It measured **slower** (3 $\times$ 8 2t: 16.4 $\rightarrow$ 17.5 μ s), and per-bundle tracing shows why:

	b_0	b_1	b_2	b_3
units	87	30	8	15
entries	824	828	674	474
busy μ s	6.32	6.33	5.49	3.75
ns/entry	7.7	7.6	8.1	7.9

The Ω -balanced 3 $\times$ 8 bundles (traced per-bundle busy times, medians over 42 000 calls). The Ω metric equalised decomposition volume, but sweep time is ≈ 7.5 ns/entry **uniformly** across wildly different degree mixes – so time tracks entry counts, and the Ω split unbalanced them (worker split 1652 : 1148 entries $\rightarrow$ busy 12.65 vs 9.24 μ s). The 65 : 35 store skew was real but time-irrelevant: scatter stores are a minor share of a uniform per-entry cost.

With the change reverted (bit-identical results re-verified; all baselines back to `scratch` within noise), the 3 $\times$ 8 two-thread call decomposes, measured component by component: shell ≈ 2.4 μ s + fork/join ≈ 0.9 μ s + sweep wall ≈ 12.7 μ s (the slower worker's busy time) ≈ 16.0 μ s ✓ (measured 16.4). The sweep's own efficiency is $\approx 1.5\times$: the per-entry cost **rises** $\approx 13\%$ at two threads (7.6 vs 6.7 ns/entry – two cores streaming the same read-only tables through shared caches), and the rest of the gap to $2\times$ is the serial shell and the fork/join round-trip. None of these respond to bundle balancing.

What would actually help (quantified levers, none of them bundling)

small shapes ($\leq 8\times 3$)	+2.6 μ s/call today	the deficit is per-fork coordination + flatten/dispatch. The fold path already amortises forks over many calls (its 3 $\times$ 8 batch reaches $1.45\times \rightarrow 3.14\times$ at 2t/8t); single-call users pay it. A cheaper entry into the sweep – not a balancing change. shell ≈ 2.4 + fork/join ≈ 0.9 are serial floor; the sweep's $\approx 13\%$ per-worker slowdown at 2t (shared-cache streaming) is a memory-locality question, not a partition question.
large shapes (3 $\times$ 8–2 $\times$ 12)	+5 μ s/call today	
batch ≥ 4 threads	already $\approx$ ideal	
		4×10 at 8 threads runs $5.85\times$ of 5.9 possible – nothing to recover.

What survives this investigation. (1) The entry-count bundling was already optimal; the rejected Ω experiment and its per-bundle tracing are the proof. (2) A new regression guard: `commutator_is_bit_identical_across_thread_counts` asserts bit-identical results at 1/2/4/8 threads on 2 $\times$ 12, 3 $\times$ 8, 4 $\times$ 6 – it passed before, during, and after the experiment, and stays in the suite. (3) The two deficits are now measured, named, and bounded: per-fork coordination $\approx 0.9\text{--}1.4$ μ s (small shapes) and shell + per-worker cache slowdown (large shapes). Any future optimisation should target one of those two numbers – not the bundle boundaries.